

UNIVERSIDADE FEDERAL DE PELOTAS
Centro de Desenvolvimento Tecnológico
Curso de Bacharelado em Ciência da Computação



Trabalho de Conclusão de Curso

Uma DSL embutida em Haskell para programação Prolog

Ari Vitor da Silva Lazzarotto

Pelotas, 2023

Ari Vitor da Silva Lazzarotto

Uma DSL embutida em Haskell para programação Prolog

Trabalho de Conclusão de Curso apresentado ao Centro de Desenvolvimento Tecnológico da Universidade Federal de Pelotas, como requisito parcial à obtenção do título de Bacharel em Ciência da Computação.

Orientador: Prof. Dr. André Rauber Du Bois

Pelotas, 2023

Universidade Federal de Pelotas / Sistema de Bibliotecas
Catalogação na Publicação

L432d Lazzarotto, Ari Vitor da Silva

Uma DSL embutida em Haskell para programação
Prolog / Ari Vitor da Silva Lazzarotto ; André Rauber Du
Bois, orientador. — Pelotas, 2023.

43 f. : il.

Trabalho de Conclusão de Curso (Graduação em Ciência
da Computação) — Centro de Desenvolvimento
Tecnológico, Universidade Federal de Pelotas, 2023.

1. Prolog. 2. DSL. 3. Haskell. 4. Programação lógica. 5.
Programação funcional. I. Du Bois, André Rauber, orient. II.
Título.

CDD : 005

Ari Vitor da Silva Lazzarotto

Uma DSL embutida em Haskell para programação Prolog

Trabalho de Conclusão de Curso aprovado, como requisito parcial, para obtenção do grau de Bacharel em Ciência da Computação, Centro de Desenvolvimento Tecnológico, Universidade Federal de Pelotas.

Data da Defesa: 15 de maio de 2023

Banca Examinadora:

Prof. Dr. André Rauber Du Bois (orientador)
Doutor em Computação pela Heriot-Watt University.

Prof. Dr. Renata Hax Sander Reiser.
Doutora em Computação pela Universidade Federal do Rio Grande do Sul.

Prof. Dr. Rafael Burlamaqui Amaral.
Doutor em Computação pela Universidade Federal Fluminense.

Dedico esta conquista, principalmente aos meus pais, cujo apoio foi fundamental em minha jornada acadêmica. Sem o seu apoio, nada disto seria possível...

AGRADECIMENTOS

Agradeço aos meus amigos e familiares, que me apoiaram e motivaram a seguir em frente, mesmo nos momentos mais difíceis. Agradeço por todas as palavras de encorajamento, abraços calorosos e orações que me ajudaram a superar os obstáculos ao longo do caminho.

Agradeço ao meu orientador, André, pela paciência e dedicação ao longo deste trabalho. Seu comprometimento com foi essencial para que eu pudesse alcançar os resultados desejados.

Agradeço também a todos os meus professores que contribuíram para a minha formação acadêmica. Agradeço por todo o conhecimento transmitido, pelos desafios propostos e pelo incentivo constante a jamais desistir. Espero que este trabalho possa ser uma forma de retribuir um pouco de tudo o que aprendi com vocês.

Por fim, expresso minha gratidão à instituição pública onde estudo e seus colaboradores, por proporcionar a estrutura necessária para minha formação acadêmica. Espero que este trabalho possa contribuir para o avanço do conhecimento em minha área de estudo e para o desenvolvimento da instituição.

A tecnologia é a força mais poderosa do nosso tempo. Ela pode ser a maior benção ou a maior maldição da humanidade. — PHIL LANE JR

RESUMO

LAZZAROTTO, Ari Vitor da Silva. **Uma DSL embutida em Haskell para programação Prolog**. Orientador: André Rauber Du Bois. 2023. 43 f. Trabalho de Conclusão de Curso (Ciência da Computação) – Centro de Desenvolvimento Tecnológico, Universidade Federal de Pelotas, Pelotas, 2023.

Os diferentes paradigmas de programação existentes se fazem necessários para resolução de problemas distintos, onde cada paradigma tem sua principal área de aplicação e suas vantagens e desvantagens ao ser aplicado na área em questão. Mas, ao unirmos dois paradigmas, em um mesmo programa, obtemos uma maior flexibilidade e uma maior capacidade de resolução de problemas, pois podemos utilizar as vantagens de cada paradigma em conjunto, e não apenas um deles isoladamente. Neste trabalho, foi desenvolvido uma DSL (linguagem de domínio específico) em **Haskell**, para interpretação de programas **Prolog**, utilizando dois paradigmas de programação, o lógico e o funcional, mas que pertencem ao mesmo estilo de programação, o declarativo. Deste modo, tentamos minimizar as desvantagens de utilizar os paradigmas em conjunto, e espera-se maximizar as vantagens.

Palavras-chave: Prolog. Haskell. Programação Funcional. DSL.

ABSTRACT

LAZZAROTTO, Ari Vitor da Silva. **A Haskell-embedded DSL for Prolog programming**. Advisor: André Rauber Du Bois. 2023. 43 f. Undergraduate Thesis (Computer Science) – Technology Development Center, Federal University of Pelotas, Pelotas, 2023.

The different existing programming paradigms are necessary to solve specific problems, where each paradigm has its main application area and advantages and disadvantages when applied in the specific area. However, when we combine two paradigms in the same program, we obtain greater flexibility and a greater capacity to solve problems, as we can use the advantages of each paradigm together, and not just one of them in isolation. In this work, a Domain-Specific Language (DSL) was developed in **Haskell** for interpreting **Prolog** programs, using two programming paradigms, logical and functional, which belong to the same declarative programming style. In this way, we attempted to minimize the disadvantages of using the paradigms together, and to maximize the advantages.

Keywords: Prolog. Haskell. Functional Programming. DSL.

LISTA DE FIGURAS

Figura 1	Exemplo de programa escrito na linguagem imperativa C (Marcos Castro, 2015).	16
Figura 2	Exemplo de programa escrito na linguagem funcional Haskell (André Du Bois, 2022).	17
Figura 3	Exemplo da estrutura de um programa Prolog	19
Figura 4	Exemplo do algoritmo de unificação (FERNÁNDEZ, 2004).	21
Figura 5	Representação das etapas da resolução SLD (FERNÁNDEZ, 2004).	23
Figura 6	Programa P (FERNÁNDEZ, 2004).	23
Figura 7	Árvore SLD para a consulta : $-likes(Z, prolog)$ usando as cláusulas do programa P (FERNÁNDEZ, 2004).	24
Figura 8	Árvore SLD para a consulta : $-likes(Z, prolog)$ usando o programa P' (FERNÁNDEZ, 2004).	25
Figura 9	Árvore SLD para a consulta : $-likes(Z, painting)$ (FERNÁNDEZ, 2004).	25
Figura 10	Programa Prolog composto apenas de fatos(átomos) que expressa uma relação entre pais e filhos.	27
Figura 11	Programa Prolog da Figura 10 escrito na sintaxe aceita pela DSL desenvolvida.	28
Figura 12	Exemplo de consulta no Prolog , utilizando o programa da Figura 10.	28
Figura 13	Exemplo de uso da função <code>queryResult</code> , consultando o programa da Figura 11.	28
Figura 14	Exemplo de um programa com uma função em Prolog	29
Figura 15	Consulta Prolog para o programa da Figura 14.	29
Figura 16	Programa da Figura 14 implementado na sintaxe da DSL desenvolvida.	29
Figura 17	Primitivas da DSL desenvolvida.	30
Figura 18	Cabeçalho das funções <code>unify</code> e <code>substituteAll</code>	30
Figura 19	Função principal da DSL, implementa a resolução SLD do algoritmo do prolog.	31
Figura 20	Cabeçalho das funções <code>queryResult</code> e <code>interpret</code>	32
Figura 21	Resultado para o exemplo da Figura 16.	32
Figura 22	Resultado para o mesmo exemplo da Figura 16, passando um argumento diferente.	32
Figura 23	Consulta realizada no exemplo 1.	42
Figura 24	Consulta realizada no exemplo 2.	42
Figura 25	Consulta realizada no exemplo 3.	42

Figura 26	Consulta realizada no exemplo 4.	43
Figura 27	Consulta realizada no exemplo 5.	43
Figura 28	Consulta realizada no exemplo 6.	43
Figura 29	Consulta realizada no exemplo 7.	43

LISTA DE ABREVIATURAS E SIGLAS

DSL	Domain Specific Languages
GHCI	Glasgow Haskell Compiler
MGU	More General Unificator
NUMA	Non-Uniform Memory Access
SIMD	Single Instruction Multiple Data
SLD	Selective Linear Definite clause resolution
SMP	Symmetric Multi-Processor
SPMD	Single Program Multiple Data

SUMÁRIO

1	INTRODUÇÃO	14
2	PROGRAMAÇÃO DECLARATIVA	16
2.1	Programação funcional: Haskell	16
2.2	Programação lógica: Prolog	18
2.2.1	Algoritmo do Prolog: Unificação	19
2.2.2	Algoritmo do Prolog: O princípio da resolução	21
2.2.3	Algoritmo do Prolog: Resolução SLD	22
2.2.4	Algoritmo do Prolog: Busca na árvore SLD	24
3	DESENVOLVIMENTO	27
3.1	Introdução	27
3.2	Utilização	27
3.3	Implementação	29
4	CONCLUSÃO	33
	REFERÊNCIAS	35
	APÊNDICE A EXEMPLOS	37
	APÊNDICE B CONSULTAS	42

1 INTRODUÇÃO

Os diferentes paradigmas de programação existentes se fazem necessários para resolução de problemas específicos, onde cada paradigma tem sua principal área de aplicação e suas vantagens e desvantagens a ser aplicado em tal área. Porém, o que acontece se unirmos dois paradigmas em um mesmo programa? Este é o conceito da Programação Lógica Funcional (ANTOY; HANUS, 2010).

Linguagens funcionais fornecem estratégias de avaliação eficientes e orientadas por demanda que suportam estruturas infinitas, enquanto linguagens lógicas fornecem não-determinismo e predicados com vários modos de entrada/saída que oferecem reutilização de código. Ambos os tipos de linguagens têm motivações semelhantes, mas fornecem recursos diferentes, extremamente úteis para o desenvolvimento de software, sendo de grande interesse uni-los em uma única linguagem (ANTOY, 2005). Tendo isto em vista, este projeto propõe o desenvolvimento de uma DSL, onde o programador pode escrever um programa na linguagem de programação lógica **Prolog**, como uma estrutura de dados da linguagem de programação funcional **Haskell** e executá-lo dentro do programa **Haskell**. Assim, obtendo as principais vantagens dos dois paradigmas dentro de um mesmo programa.

A evolução das linguagens de programação consiste na introdução gradual de abstrações, ocultando o hardware do computador e os detalhes da execução do programa. As linguagens de montagem introduzem instruções mnemônicas e rótulos simbólicos para camuflar códigos de máquina e endereços. Fortran introduz arrays e expressões em notação matemática padrão para esconder registradores. As linguagens do tipo Algol introduzem instruções estruturadas para ocultar gotos e *Jump Labels*. As linguagens orientadas a objetos introduzem níveis de visibilidade e encapsulamento para esconder a representação de dados e o gerenciamento de memória. Nessa linha, linguagens declarativas, cujos representantes mais proeminentes são linguagens funcionais e lógicas, que ocultam a ordem de avaliação removendo atribuições e outras instruções de controle (ANTOY; HANUS, 2010).

A programação lógica funcional é a combinação, em uma única linguagem de programação, dos paradigmas de programação funcional e programação lógica. Este

estilo de programação é incorporado por várias linguagens de programação, incluindo Curry e Mercury (HANUS; KUCHEN; MORENO-NAVARRO, 1995). Entre as vantagens da programação lógica funcional podemos citar: escrever programas com especificações claras que abstraem da ordem de avaliação e o reaproveitamento de código. Claro, há um preço a pagar por esses recursos avançados. Lidar com incógnitas requer a consideração de diferentes alternativas durante o tempo de execução (ANTOY; HANUS, 2010).

Outros trabalhos, geralmente, implementam interpretadores em **Haskell** ou em **Prolog**. Então os dois paradigmas ficam separados um do outro. Há várias ferramentas que o **Haskell** oferece que tornam a implementação desta DSL muito mais simples, por exemplo: O polimorfismo de tipos onde quanto mais genérica a função, mais chances de ela ser reutilizada, o que contribui ainda mais para o reaproveitamento de código. Também, na maioria dos casos de recursão em cima de listas, pode-se resolver usando algumas funções de alta ordem pré-definidas em **Haskell**. Usando abstrações lambda podemos escrever expressões que representam funções sem nome. Uma boa aplicação para as expressões lambda, é quando se necessita definir funções que são argumentos para funções de alta ordem. Como a composição de funções é muito usada em **Haskell** existe um operador específico para composição de funções (THOMPSON, 1996).

Deste modo, o objetivo deste trabalho é desenvolver uma DSL que permita unir diferentes paradigmas de linguagens de programação. A ideia é que ela permita tirar vantagem do melhor das duas linguagens. A linguagem **Prolog**, que proporcionará o frontend dessa solução, está presente nos mais diversos ambientes associados com inteligência artificial e linguística computacional, sendo ela uma linguagem declarativa, que significa que em vez de o programa estipular a maneira de chegar à solução, passo a passo (como linguagens procedurais ou imperativas), limita-se a fornecer uma descrição do problema que se pretende computar. Utiliza uma base de dados de fatos e relações lógicas (regras) que representam o domínio relacional do problema a ser resolvido (WIELEMAKER et al., 2012).

Nos próximos capítulos serão apresentadas breves introduções ao **Haskell** e a programação funcional e ao **Prolog** e a programação lógica. Na seção sobre programação funcional, será discutido a necessidade de diferentes paradigmas e linguagens de programação, alternativas ao **Haskell**, suas principais áreas de aplicação e sua evolução e relevância científica nos últimos anos. Já na seção sobre programação lógica, não nos atentaremos a sua fundamentação matemática e detalhes da definição da lógica proposicional, mas discutiremos a definição e utilização de suas fórmulas lógicas, tipos de dados e arquitetura do Algoritmo do **Prolog**. Por fim, será apresentado o desenvolvimento de uma versão simplificada do algoritmo, escrita na linguagem funcional **Haskell**, capaz de interpretar programas **Prolog**.

2 PROGRAMAÇÃO DECLARATIVA

2.1 Programação funcional: Haskell

As linguagens de programação imperativas popularmente conhecidas compartilham uma ideia de design comum: tornar o uso mais fácil e eficiente da arquitetura Von Neumann, basicamente fornecendo abstrações para armazenamento de dados e programas, para que os programadores não necessitem manter o controle de endereços de memória, e estruturas de controle, que tornam os programas mais legíveis e fáceis de escrever. Embora o nível de abstração fornecido pelas linguagens imperativas varie, eles refletem a arquitetura da máquina subjacente, o que facilita o ato de gerar código para os compiladores .

```
1  /*
2   Implementação do QuickSort em C
3   Autor: Marcos Castro - www.GeeksBR.com
4  */
5
6  #include <stdio.h>
7  #include <stdlib.h> // srand, rand
8  #include <time.h> // time
9
10 void troca(int vet[], int i, int j){
11     int aux = vet[i];
12
13     vet[i] = vet[j];
14     vet[j] = aux;
15 }
16
17 int particiona(int vet[], int inicio, int fim){
18     int pivo, pivo_indice, i;
19
20     pivo = vet[fim];
21     pivo_indice = inicio;
22     for(i = inicio; i < fim; i++){
23         if(vet[i] <= pivo){
24             troca(vet, i, pivo_indice);
25             pivo_indice++;
26         }
27     }
28     troca(vet, pivo_indice, fim);
29     return pivo_indice;
30 }
31
32 int particiona_random(int vet[], int inicio, int fim){
33     int pivo_indice = (rand() % (fim - inicio + 1)) + inicio;
34
35     troca(vet, pivo_indice, fim);
36     return particiona(vet, inicio, fim);
37 }
38
39 void quick_sort(int vet[], int inicio, int fim){
40     if(inicio < fim){
41         int pivo_indice = particiona_random(vet, inicio, fim);
42
43         quick_sort(vet, inicio, pivo_indice - 1);
44         quick_sort(vet, pivo_indice + 1, fim);
45     }
46 }
47
48 int main(){
49     int vet[] = {25,40,55,20,44,35,38,99,10,65,50};
50     int tam_vet = sizeof(vet) / sizeof(int);
51     int i;
52
53     srand(time(NULL));
54     quick_sort(vet, 0, tam_vet - 1);
55     for(i = 0; i < tam_vet; i++){
56         printf("%d ", vet[i]);
57     }
58     return 0;
59 }
```

Figura 1 – Exemplo de programa escrito na linguagem imperativa C (Marcos Castro, 2015).

No entanto, ao desenvolver aplicativos de software, a eficiência nem sempre é uma prioridade. Dependendo da aplicação, propriedades como baixo custo de manutenção, fácil depuração, ou correção formalmente demonstrável, pode ter prioridade mais alta. Por exemplo, em domínios críticos de segurança (como medicina, telecomunica-

```
quickSort :: Ord a => [a] -> [a]
quickSort [] = []
quickSort (x:xs) = quickSort [a | a<-xs, a<=x] ++ [x] ++ quickSort [a | a<-xs, a>x]
```

Figura 2 – Exemplo de programa escrito na linguagem funcional **Haskell** (André Du Bois, 2022).

ções ou sistemas de transporte) é importante desenvolver programas cuja correção possa ser certificada (ou seja, provado formalmente). As linguagens declarativas, e em particular as linguagens funcionais, podem ser uma boa alternativa neste caso. Programas em linguagens funcionais são geralmente mais curtos, mais fáceis de entender, mais fáceis de projetar, depurar e manter do que suas contrapartes imperativas.

As Figuras 1 e 2 demonstram isto, de modo que o mesmo algoritmo, é expressado em um programa muito maior na implementação imperativa, do que na implementação funcional. Por este motivo, linguagens de programação funcionais estão se tornando cada vez mais populares. Os principais domínios de aplicação das linguagens funcionais são a inteligência artificial (por exemplo, para a implementação de sistemas especialistas), processamento de texto (por exemplo, o editor UNIX Emacs é implementado em LISP), linguagem natural, telefonia, interfaces gráficas, provadores de teoremas e assistentes de prova. LISP, que foi introduzido por McCarthy na década de 1950, é considerado a primeira linguagem de programação funcional. A sintaxe do LISP é baseada em listas (como o nome da linguagem sugere: LISt Processing), onde os elementos atômicos são números e caracteres.

Nos últimos anos, surgiram várias linguagens funcionais, que mudaram radicalmente a sintaxe e introduziram sistemas de tipos sofisticados com recursos de inferência de tipos. As linguagens funcionais modernas são fortemente tipadas (o que garante que nenhum tipo de erro pode surgir em tempo de execução), com escopo estático e gerenciamento de memória integrado. ML¹ e **Haskell**² são exemplos disso, porém diferem em sua semântica: ML é estrito, enquanto **Haskell** é preguiçoso, o que significa que apenas cálculos que são necessários para avaliar o programa serão realizados (FERNÁNDEZ, 2004). Outra diferença é que ML inclui alguns recursos imperativos (por razões de eficiência), enquanto **Haskell** é uma linguagem funcional pura gozando de transparência referencial: a avaliação de uma expressão depende apenas de seus componentes e, portanto, é independente de seu contexto.

A característica comum de todas as linguagens funcionais é que os programas consistem em funções (como na teoria matemática das funções, que é diferente da noção de função usada em linguagens imperativas). Uma função matemática é um

¹ML é uma linguagem funcional impura, pode ser interpretada online gratuitamente, em <https://sosml.org>

²**Haskell** é uma linguagem funcional pura disponível gratuitamente na Internet: seu interpretador pode ser baixado, incluindo o relatório de idioma e o manual do usuário, de <https://www.haskell.org>

mapeamento entre elementos de dois conjuntos. Por exemplo, a função $f : A \rightarrow B$ associa cada elemento do conjunto A (o domínio da função) com no máximo um elemento no conjunto B (o contradomínio da função). Dizemos que a função f tem tipo $A \rightarrow B$ (ou f é do tipo $A \rightarrow B$). Linguagens funcionais são declarativas, para definir uma função, os programadores podem dar uma definição direta do mapeamento ou defini-lo em termos de outras funções previamente definidas (ou seja, funções que já foram especificadas pelo programador, retiradas de bibliotecas ou fornecidas como primitivas de linguagem).

2.2 Programação lógica: Prolog

Nas linguagens de programação lógica, em resumo, um programa consiste em fórmulas que descrevem um problema. A execução de um programa é um processo de busca de provas, durante a qual são geradas as soluções para o problema. Uma vez que os programas são apenas descrições de problemas, este é um estilo de programação baseado em conhecimento, que tem muitas aplicações em inteligência artificial (por exemplo, para construir sistemas especialistas).

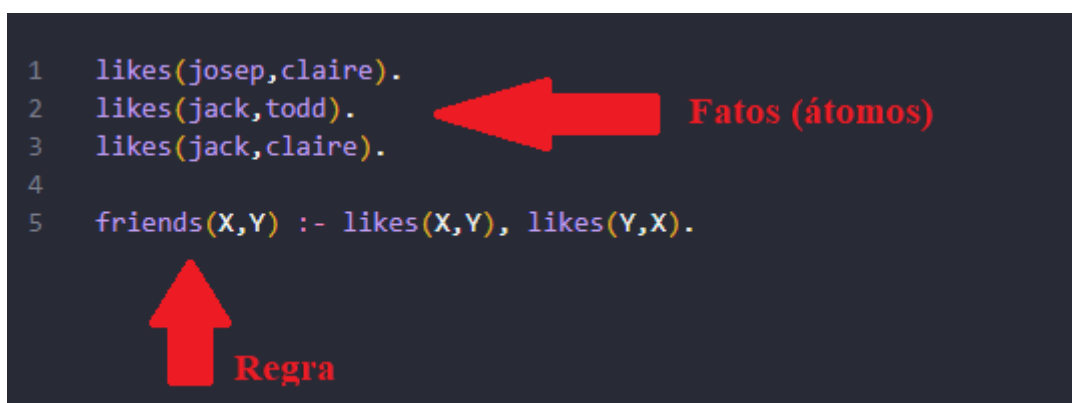
A linguagem de programação lógica é muito poderosa. O mesmo formalismo pode ser usado para especificar um problema, escrever um programa e provar as propriedades do programa. O mesmo programa pode ser usado de muitas maneiras diferentes, divergindo no tipo de lógica que é usada para a descrição do problema e o método empregado para encontrar provas.

A linguagem de programação lógica mais conhecida é o **Prolog** (<https://www.swi-prolog.org>), que é baseado em lógica de primeira ordem e usa o Princípio da Resolução para construir provas. Na realidade, lógica e resolução de primeira ordem são muito gerais para serem usadas diretamente como linguagem (FERNÁNDEZ, 2004), mas na década de 1970, Kowalski, Colmerauer e Roussel definiram e implementaram uma restrição adequada, baseada no fragmento clausal clássico de primeira ordem lógica. Isso resultou na primeira versão do **Prolog** (ROUSSEL, 1975).

Atualmente, existem várias versões do **Prolog**. A estrutura básica foi enriquecida para torná-la mais eficiente e fácil de usar. As extensões incluem dados primitivos, tipos para números inteiros e reais, técnicas avançadas de otimização, manipulação de arquivos facilidades, interfaces gráficas, mecanismos de controle, etc. Alguns desses recursos são não-declarativos, e os programas que os utilizam são chamados impuros porque, para atingir eficiência, a descrição do problema e os detalhes de implementação são misturados (ou seja, o "quê" e o "como" são misturados). Linguagens de programação de lógica de restrição, que foram desenvolvidos a partir do **Prolog**, alcançam eficiência incorporando provas otimizadas e métodos de pesquisa para domínios específicos (FERNÁNDEZ, 2004).

O **Prolog** funciona utilizando lógica de predicados(ou lógica proposicional) (DI-NUCCI et al., 2016), a partir de uma série de formulas lógicas chamadas cláusulas de Horn (MONARD; BARANAUSKAS, 2003). Estas cláusulas possuem um lado esquerdo e direito($h \leftarrow p_1, p_2, \dots, p_n$), onde o lado esquerdo (h) é verdadeiro se e somente se, o que há no lado direito ($p_1, p_2, \dots, p_n$), sendo um ou n requisitos, forem simultaneamente verdadeiros. Cláusulas que não possuem um lado direito(algo a ser provado) são denominados átomos, estes são fatos, verdades absolutas, que permitem a definição de predicados, entradas do programa utilizados na prova das cláusulas.

A estrutura de um programa **Prolog** é exibida na Figura 3, onde primeiro são definidos 3 fatos em uma base de conhecimento, que define quais indivíduos gostam uns dos outros, e uma regra capaz de inferir quem é o amigo de um indivíduo, baseado nos fatos da base de conhecimento. No exemplo, para um indivíduo ser considerado amigo do outro, os dois devem gostar um do outro.



```

1 likes(josep,claire).
2 likes(jack,todd).
3 likes(jack,claire).
4
5 friends(X,Y) :- likes(X,Y), likes(Y,X).
```

Fatos (átomos)

Regra

Figura 3 – Exemplo da estrutura de um programa **Prolog**.

O **Prolog** tem apenas um tipo de dados — o termo. O termo mais simples é um átomo. Os átomos não têm um significado inerente. Com consultas, o usuário do programa pode fazer perguntas ao **Prolog** sobre a informação armazenada na base de conhecimento criada. Através do procedimento de Backtracking (GINSBERG, 1993) o **Prolog** pesquisa o valor verdadeiro de diferentes predicados, verificando se eles estão corretos ou não. Até chegar ao destino esperado (*goal*), ele tenta voltar atrás, quando o destino é encontrado, ele para. Algo importante a ser comentado é que o **Prolog** é case sensitive, isto é, a linguagem difere letras minúsculas de maiúsculas, onde: Strings que começam com letra minúscula são CONSTANTES, os átomos; Já as strings que começam com letra maiúscula(ou underscore) são VÁRIAVEIS.

2.2.1 Algoritmo do Prolog: Unificação

A etapa de unificação é um processo de simplificação que utiliza o princípio da resolução onde uma substituição pode ser realizada em alguns casos para verificar

se duas expressões são idênticas, analisando se a função é a mesma e se o número de termos é coincidente. O conjunto de termos de equivalência válidos que expressa essa substituição é denominado unificador mais geral (*Most General Unifier* - *MGU*) (SNYDER, 2001).

O algoritmo de Martelli e Montanari (MARTELLI; MONTANARI, 1982) encontra o unificador mais geral para um problema de unificação se existe uma solução; caso contrário, ele falha, indicando que não há soluções. Para encontrar o unificador mais geral para um problema de unificação, o algoritmo simplifica o conjunto de equações, até que uma substituição seja gerada ou uma falha seja detectada. As regras de simplificação aplicam-se a conjuntos de equações e produzem novos conjuntos de equações ou uma falha. Possui como entrada um conjunto finito de equações entre termos (ou seja, um problema de unificação), então, simplifica-se o problema de unificação em um mais simples, utilizando regras de transformação, ou produzindo uma falha, gera na saída uma substituição que é MGU para este problema, ou um fracasso.

O algoritmo de unificação aplica as regras de transformação de forma não-determinística, até que nenhuma regra possa ser aplicada ou uma falha seja encontrada, como este trabalha com conjuntos de equações, a ordem em que elas aparecem no problema da unificação não é importante (FERNÁNDEZ, 2004).

Um problema de unificação U é um conjunto de equações entre termos contendo variáveis. Usaremos a notação $s_1 = t_1, \dots, s_n = t_n$ para um problema de unificação consistindo de n pares de termos para unificar.

Uma solução para o problema de unificação U , também chamado de unificador para o problema U , é uma substituição σ tal que quando aplicamos σ a todos os termos nas equações em U obtemos identidades sintáticas: Para cada equação $s_i = t_i$ em U , os termos $s_i\sigma$ e $t_i\sigma$ coincidem.

Uma substituição, σ , é um unificador mais geral (mgu) de um conjunto de expressões E se unifica E , e para qualquer unificador, ω , de E , existe um unificador, λ , tal que $\omega = \sigma \circ \lambda$.

A ideia é que σ seja menos específico que (tecnicamente, não mais específico que) ω , ou seja, podemos substituir algumas das variáveis de σ e obter ω . Observe que pode haver mais de um unificador mais geral, mas essas substituições são as mesmas, exceto pela renomeação de variáveis.

Se existe um unificador para o problema $s = t$, dizemos que s e t são unificáveis. Um unificador σ para U é dito mais geral se qualquer outro unificador para o problema U pode ser obtido como uma instância de σ .

Embora possa haver muitas substituições diferentes que são unificadores mais gerais para um problema U , são todas equivalentes a renomeação de variáveis. Em outras palavras, se um problema de unificação U tem um unificador, então há um unificador único mais geral que as renomeações, o que justifica a referência ao MGU

para U (FERNÁNDEZ, 2004).

- (1) $f(s_1, \dots, s_n) = f(t_1, \dots, t_n), E \rightarrow s_1 = t_1, \dots, s_n = t_n, E$
- (2) $f(s_1, \dots, s_n) = g(t_1, \dots, t_m), E \rightarrow failure$
- (3) $X = X, E \rightarrow E$
- (4) $t = X, E \rightarrow X = t, E$
if t is not a variable
- (5) $X = t, E \rightarrow X = t, E\{X \mapsto t\}$
if X is not in t and X occurs in E
- (6) $X = t, E \rightarrow failure$
if X occurs in t and $X \neq t$

Figura 4 – Exemplo do algoritmo de unificação (FERNÁNDEZ, 2004).

2.2.2 Algoritmo do Prolog: O princípio da resolução

O algoritmo de unificação apresentado na Figura 4 aplica as regras de transformação de forma não determinística até que nenhuma regra possa ser aplicada ou surja uma falha. Observe que estamos trabalhando com conjuntos de equações, portanto, a ordem em que aparecem no problema de unificação não é importante.

O teste no caso (6) da Figura 4 é chamado de verificação de ocorrência; por exemplo: $X = f(X)$ falha. Esse teste é demorado, e por esta razão em alguns sistemas não é implementado. Se o algoritmo terminar sem falhas, obtemos uma substituição, que é o unificador mais geral do conjunto inicial de equações.

Observe que as regras (1) e (2) também se aplicam a equações entre constantes (ou seja, 0-ário símbolos de função). No primeiro caso, se $n = 0$, então a equação $f = f$ é excluída, e no segundo caso, o problema não tem solução (o algoritmo para com falha).

A etapa de resolução é baseada na refutação. A fim de resolver uma consulta : $-A_1, \dots, A_n$ em relação a um conjunto de cláusulas P , a resolução busca mostrar a semântica operacional do **Prolog** leva a uma contradição. Ou seja, $P, A_1, \dots, A_n$. A negação dos literais no objetivo é adicionada ao programa P ; se surge uma contradição, então sabemos que P fez implicar os literais na consulta.

Uma contradição é obtida quando um literal e sua negação são declarados ao mesmo tempo. Se uma contradição não surgir diretamente do programa e do objetivo, novas cláusulas serão derivadas por resolução, e o processo continuará até que surja uma contradição ou mais deduções não possam ser feitas (a busca pode continuar para sempre). As cláusulas derivadas são chamadas de desarmes. A geração de resolutores pode ser descrita usando uma restrição do princípio de resolução chamado resolução SLD; O Prolog é baseado na resolução SLD (FERNÁNDEZ, 2004).

2.2.3 Algoritmo do Prolog: Resolução SLD

Vamos considerar primeiro um caso simples, onde na consulta há apenas um átomo. Se nós temos uma pergunta : $-p(u1, \dots, un)$ e uma cláusula de programa (onde renomeamos as variáveis, caso necessário, para que todas as variáveis na cláusula sejam diferentes daquelas na consulta) $p(t1, \dots, tn) : -S1, \dots, Sm$ tal que o problema de unificação $p(t1, \dots, tn) = p(u1, \dots, un)$ tem um MGU σ , então obtemos um resolvente : $-S1\sigma, \dots, Sm\sigma$. No caso geral, a consulta pode ter vários literais. A resolução SLD do **Prolog** gera um resolvente usando o primeiro literal na meta.

Um resolvente para a meta : $-A1, \dots, Ak$ em relação a um determinado programa P é gerado pela unificação do primeiro átomo no objetivo (ou seja, A1) e o cabeçalho de uma cláusula de programa (possivelmente renomeada). Mais precisamente, se houver uma cláusula em P da forma $A'1 : -S1, \dots, Sm$ tal que A'1 e A1 são unificáveis com MGU σ , então obtemos um resolvente : $-S1\sigma, \dots, Sm\sigma, A2\sigma, \dots, Ak\sigma$.

Em outras palavras, o resolvente é gerado substituindo na meta o primeiro átomo que unifica, e aplicando o unificador para todos os átomos na nova meta. Observe que quando computamos um resolvente usando um fato (ou seja, quando $m = 0$), o átomo simplesmente desaparece da consulta. Um resolvente vazio indica uma contradição, que denotaremos pelo símbolo $\diamond$.

A resolução SLD é dividida em etapas, onde cada uma delas calcula um resolvente entre o primeiro átomo do último resolvente obtido e uma cláusula no programa. É por essa razão que esta forma particular de resolução é chamada de resolução SLD: 'S' significa regra de seleção: Uma regra de computação fixa é aplicada para selecionar um átomo em particular para resolver no objetivo. O **Prolog** sempre seleciona o literal mais à esquerda no objetivo. 'L' significa linear: indica que cada etapa de resolução usa o mais recente resolvente (para começar, ele usa a consulta dada) e uma cláusula do programa. O **Prolog** usa as cláusulas do programa na ordem em que são escritas. 'D' significa definitivo: indica que todas as cláusulas do programa são definitivas.

Dado um programa e uma consulta, a ideia é continuar gerando resolventes até uma contradição ser gerada. Quando um resolvente vazio é gerado, a composição de todas as substituições aplicadas em cada etapa de resolução SLD levando a a contradição é computada. Esta também é uma substituição (lembre-se que as substituições são funções de termos a termos e composição aqui significa composição funcional). A restrição desta substituição às variáveis que ocorrem no objetivo inicial é uma resposta para a consulta inicial. Representamos cada etapa de resolução SLD graficamente como exibido na Figura 5.

Como pode haver várias cláusulas no programa que podem ser usadas para gerar um resolvente para uma determinada consulta, obtemos uma estrutura de ramificação chamada árvore de resolução SLD(ou apenas SLD-tree). A árvore SLD para uma determinada consulta tem a consulta na raiz e uma ramificação para cada resolvente

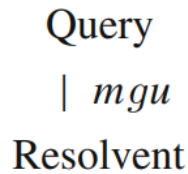


Figura 5 – Representação das etapas da resolução SLD (FERNÁNDEZ, 2004).

gerado a partir da consulta, que por sua vez se torna a raiz de uma subárvore. Cada ramificação que leva a um resolvente vazio na árvore SLD produz uma resposta. Todas as ramificações que produzem uma resposta são chamadas de ramificações de sucesso. Se uma ramificação finita não levar a um resolvente vazio, é uma falha. Uma árvore de resolução SLD pode ter várias ramificações de sucesso, ramificações de falha e também infinitas ramificações que surgem quando podemos continuar gerando resolventes, mas nunca chegar a um vazio (FERNÁNDEZ, 2004).

Considere o programa:

```
based (prolog,logic) .
based (haskell,maths) .
likes (max,logic) .
likes (claire,maths) .
likes (X,P) :- based (P,Y) , likes (X,Y) .
```

Figura 6 – Programa P (FERNÁNDEZ, 2004).

E a consulta : $\neg likes(Z, prolog)$. Usando a última cláusula, e o mgu $X \mapsto Z, P \mapsto prolog$ obtemos o resolvente : $\neg based(prolog, Y), likes(Z, Y)$. Agora, usando a primeira cláusula e a mgu $Y \mapsto logic$, obtemos a nova resolvente : $\neg likes(Z, logic)$. Finalmente, como podemos unificar este átomo com o fato $likes(max, logic)$ usando a substituição $Z \mapsto max$, obtemos um resolvente vazio.

Este é, portanto, um ramo de sucesso na árvore SLD para a consulta inicial. A composição das substituições usadas neste ramo é: $X \mapsto max, P \mapsto prolog, Y \mapsto logic, Z \mapsto max$. Portanto, a resposta para a pergunta inicial é $Z \mapsto max$. Existem outras ramificações na árvore SLD para esta consulta, mas esta é a única com sucesso. A árvore de resolução SLD para esta consulta é mostrada na Figura 7. Observe que na ramificação que leva à falha, usamos novamente a última cláusula do programa, mas renomeamos suas variáveis como X', P', Y' para evitar confusão com o uso anterior desta cláusula.

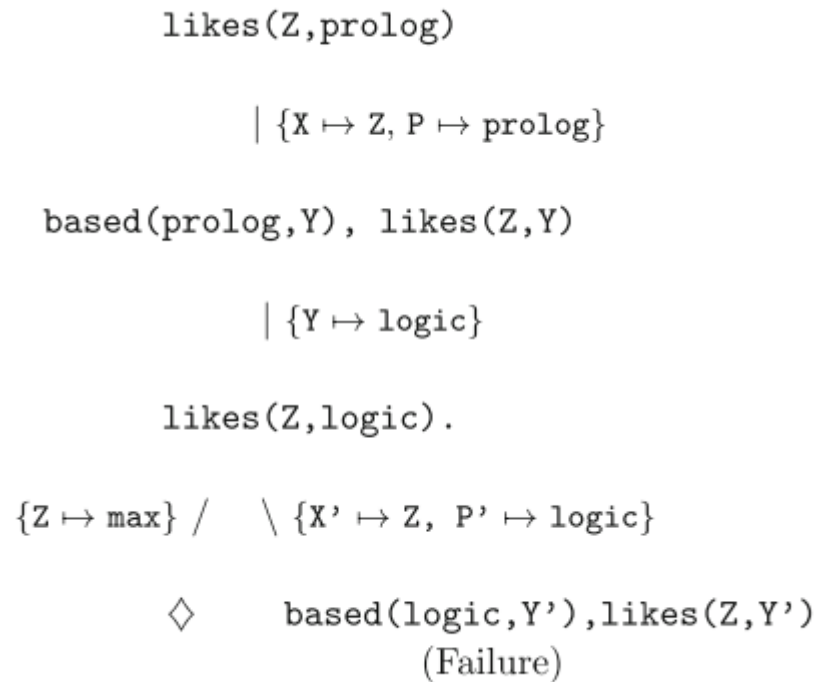


Figura 7 – Árvore SLD para a consulta : $\neg \text{likes}(Z, \text{prolog})$ usando as cláusulas do programa P (FERNÁNDEZ, 2004).

Considere agora o mesmo programa com uma cláusula adicional: $\text{likes}(\text{claire}, \text{logic})$. O novo programa será chamado P' . A árvore de resolução SLD para a mesma consulta no contexto do programa P' é mostrado na Figura 8. Finalmente, com o mesmo programa e uma consulta : $\neg \text{likes}(Z, \text{painting})$ a árvore SLD consiste em apenas um ramo (que termina em falha), como mostrado na Figura 9.

2.2.4 Algoritmo do Prolog: Busca na árvore SLD

Na última etapa, o **Prolog** constrói a árvore SLD para uma determinada consulta usando as cláusulas do programa na ordem em que elas ocorrem, em primeiro lugar; o ramo mais à esquerda da árvore SLD é gerado primeiro. Se este ramo for infinito, o **Prolog** não encontrará uma resposta mesmo que existam outras ramificações bem-sucedidas. Por isso, a ordem das cláusulas em um programa **Prolog** é muito importante.

Se durante o percurso da árvore, o **Prolog** chegar a uma folha com falha, ele voltará (em direção à raiz da árvore) para explorar os ramos restantes. Esse processo é chamado Backtracking. O **Prolog** geralmente para depois de encontrar a primeira solução para uma determinada consulta, mas retrocede e continua atravessando a árvore (sempre em uma busca em profundidade, da esquerda para a direita) se precisarmos de mais respostas (FERNÁNDEZ, 2004).

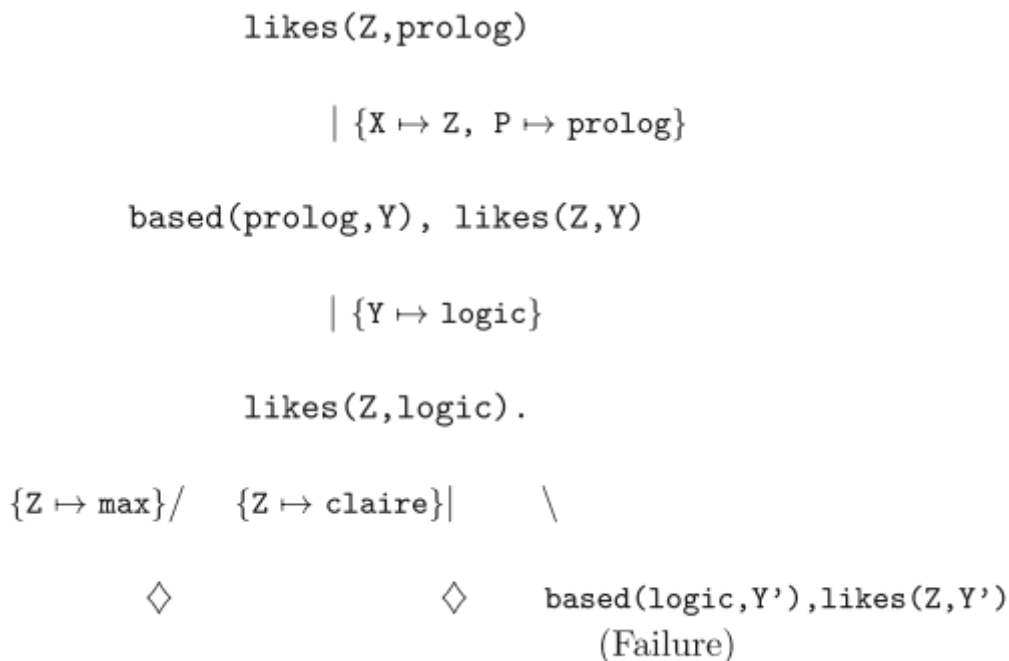


Figura 8 – Árvore SLD para a consulta : $\neg likes(Z, prolog)$ usando o programa P' (FERNÁNDEZ, 2004).

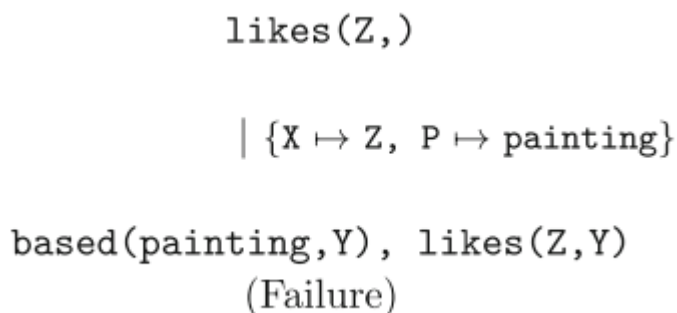


Figura 9 – Árvore SLD para a consulta : $\neg \text{likes}(Z, \text{painting})$ (FERNÁNDEZ, 2004).

Ao definir o predicado *append*.

$$append([], L, L).$$
$$append([X|L], Y, [X|Z]) : -append(L, Y, Z).$$

e a meta : $\neg append(X, [1, 2], U)$.

produz a resposta $X \mapsto [], U \mapsto [1, 2]$.

No entanto, se mudarmos a ordem das cláusulas do programa, o mesmo objetivo leva a uma computação infinita. Nesse caso, não há resposta para a consulta e, eventualmente, o interpretador exibirá uma mensagem de erro (quando todo o espaço de memória disponível é consumido). Isto porque o ramo mais à esquerda da árvore SLD é infinito.

A resolução SLD tem propriedades computacionais interessantes:

1. É uma refutação completa: dado um programa **Prolog** e um objetivo, se uma contradição pode ser derivada, então a resolução SLD irá eventualmente gerar um resolvente vazio.
2. É independente da regra de cálculo: se houver uma resposta para um objetivo, a resolução SLD irá encontrá-la qualquer que seja a regra de seleção empregada para escolher os literais resolvidos.

No entanto, a estratégia específica de percurso de uma árvore que o **Prolog** utiliza não está completa. Dentro no exemplo acima, vemos que se alterarmos a ordem das cláusulas no programa, o **Prolog** não consegue encontrar uma resposta, mesmo que um resolvente vazio possa ser gerado pela resolução SLD. O problema é que esse resolvente vazio será gerado em uma ramificação da árvore SLD que o **Prolog** não constrói. Existe uma maneira fácil de obter uma implementação completa de refutação da resolução SLD: usando busca em largura em vez de busca em profundidade. No entanto, existe um preço a pagar. Uma estratégia de busca ampla geralmente levará mais tempo para encontrar a primeira resposta. Por esta razão, esta estratégia raramente é aplicada na prática (FERNÁNDEZ, 2004).

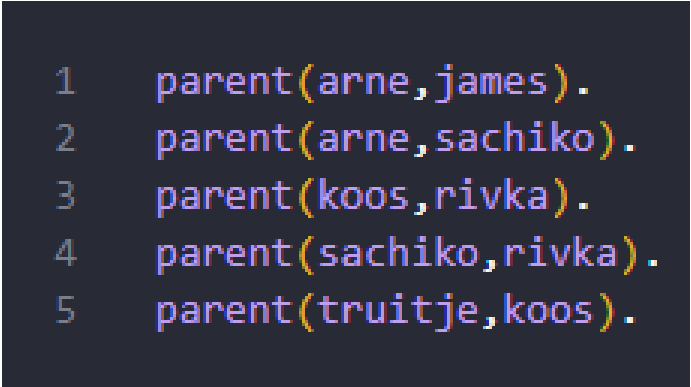
3 DESENVOLVIMENTO

3.1 Introdução

O programa desenvolvido é uma DSL, sigla que significa Linguagem de Domínio Específico. É uma linguagem de programação criada com o objetivo de resolver problemas específicos, tornando a codificação mais fácil e intuitiva.

Geralmente, as DSLs são projetadas para serem mais próximas da linguagem natural do domínio, tornando a escrita de programas mais fácil para pessoas que não são programadoras experientes. Porém neste caso, a DSL desenvolvida tem como objetivo executar programas **Prolog**, aproveitando as funcionalidades e tipos de dados da linguagem **Haskell**.

3.2 Utilização



```
1 parent(arne,james).
2 parent(arne,sachiko).
3 parent(koos,rivka).
4 parent(sachiko,rivka).
5 parent(truitje,koos).
```

Figura 10 – Programa Prolog composto apenas de fatos(átomos) que expressa uma relação entre pais e filhos.

A Figura 10 é um exemplo simples de um programa **Prolog** composto apenas por fatos, este programa, deve ser escrito na sintaxe aceita pela DSL desenvolvida, como exibido na Figura 11. Note como este exemplo é uma lista de cláusulas simples (um único termo sem um corpo), onde cada uma delas é uma função (Func: uma função com um nome e uma lista de termos como argumentos) que possui em sua lista de argumentos dois átomos.

Devemos primeiro salvar o código em um arquivo com extensão `.hs`, que é a extensão utilizada para arquivos **Haskell**. Utilizando o seu interpretador, o `GHCI`, carregamos o arquivo salvo através do comando `ghci "nome-do-arquivo".hs`. Podemos então utilizar a função `queryResult`, que realizará a consulta, neste caso, na base de dados **Prolog** exibida na Figura 11, e retornará a solução encontrada.

```
--Modern Compiler Design Example page 613
myExample1 :: Prolog
myExample1 = [
    Simple (Func "parent" [Atom "arne", Atom "james"]),
    Simple (Func "parent" [Atom "arne", Atom "sachiko"]),
    Simple (Func "parent" [Atom "koos", Atom "rivka"]),
    Simple (Func "parent" [Atom "sachiko", Atom "rivka"]),
    Simple (Func "parent" [Atom "truitje", Atom "koos"])]
```

Figura 11 – Programa **Prolog** da Figura 10 escrito na sintaxe aceita pela DSL desenvolvida.

A consulta em **Prolog** exemplificada na Figura 12, realizada no programa da Figura 10 pode ser escrita na sintaxe aceita pela DSL como mostrado na Figura 13. Esta consulta retorna `[("X","arne")]`, que representa a substituição realizada `X = arne` como a única solução para a consulta. Se não houver solução para a consulta, a função retorna uma lista vazia.

```
?- parent(X,james).
X = arne.
```

Figura 12 – Exemplo de consulta no **Prolog**, utilizando o programa da Figura 10.

```
queryResult myExample1 (Func "parent" [Var "X", Atom "james"])
```

Figura 13 – Exemplo de uso da função `queryResult`, consultando o programa da Figura 11.

Agora, considere o programa da Figura 14, que define uma função através de dois fatos diferentes. O primeiro fato `based`, define que `prolog` está ligado a `logic`, e `haskell` a `maths`. O fato `likes`, define que `max` gosta de `logic` e que `claire` gosta de `maths`. A regra `likes` (que não necessariamente precisa ter o mesmo nome do fato) compõe os dois fatos pré-estabelecidos junto ao uso de variáveis, a fim de obter uma resposta para a consulta de inferência, como exibido na Figura 15.

Esta regra é definida em termos dos fatos `based` e `likes`, onde, para satisfazer a consulta, os dois termos existentes no corpo da regra, devem ser unificados. Quando consultado se existe algum `X` que goste de `prolog`, através da base de conhecimento e da regra definida, o algoritmo infere que como `prolog` é baseado em lógica e que `max` gosta de lógica, logo, `max` gosta de `prolog`. O programa **Prolog** da Figura 14, pode ser implementado na sintaxe aceita pela DSL como exibido na Figura 16.

```

1  based(prolog,logic).
2  based(haskell,maths).
3  likes(max,logic).
4  likes(claire,maths).
5  likes(X,P) :- based(P,Y), likes(X,Y).
6

```

Figura 14 – Exemplo de um programa com uma função em **Prolog**.

```

?- likes(X,prolog).
X = max ,

```

Figura 15 – Consulta **Prolog** para o programa da Figura 14.

```

--Maribel Example page 171
myExample2 :: Prolog
myExample2 = [
  Simple (Func "based" [Atom "prolog", Atom "logic"]),
  Simple (Func "based" [Atom "haskell", Atom "maths"]),
  Simple (Func "likes" [Atom "max", Atom "logic"]),
  Simple (Func "likes" [Atom "claire", Atom "maths"]),
  Func "likes" [Var "X", Var "P"] :- [Func "based" [Var "P", Var "Y"], Func "likes" [Var "X", Var "Y"]]
]

```

Figura 16 – Programa da Figura 14 implementado na sintaxe da DSL desenvolvida.

3.3 Implementação

Para criar uma DSL, é necessário escolher uma série de primitivas que serão usadas para descrever os programas naquela linguagem. No caso do **Prolog**, as primitivas necessárias são: Predicados; As unidades básicas da programação em **Prolog**, e geralmente representam relações entre entidades do domínio em questão. Por exemplo, um predicado "mãe(X, Y)" pode ser usado para representar a relação "X é mãe de Y". Fatos; Fatos são predicados que são verdadeiros em todo o tempo de execução do programa, geralmente usados para descrever informações estáticas do domínio, como por exemplo "joão é pai de maria". Regras; Usadas para definir predicados em termos de outros predicados.

Por exemplo, uma regra "avô(X, Y) :- pai(X, Z), pai(Z, Y)" pode ser usada para definir o predicado "avô" em termos dos predicados "pai". Variáveis; São usadas para representar valores que podem ser instanciados durante a execução do programa. Por exemplo, na regra acima, as variáveis X, Y e Z são usadas para representar valores que serão instanciados de acordo com as entradas do usuário. E por fim, os Operadores; O **Prolog** possui uma série de operadores, como o operador de negação (not), o operador de disjunção (ou), o operador de conjunção (e) etc. Eles são usados para construir predicados mais complexos a partir de predicados mais simples. Como

estes operadores não são necessários para execução de programas simples, eles não serão implementados neste trabalho.

```
data Term = Var String | Atom String | Func String [Term]
    deriving (Eq, Show)
data Clause = Term :- [Term] | Simple Term
    deriving (Eq, Show)

type Prolog = [Clause]
```

Figura 17 – Primitivas da DSL desenvolvida.

Inicialmente, foram definidos os dois tipos de dados principais do **Prolog**: Termo e Cláusula. O tipo `Term` representa um termo **Prolog** e pode ser uma variável, um átomo ou uma função. O construtor `Var` representa uma variável, o construtor `Atom` representa um átomo e o construtor `Func` representa uma função, que consiste em um nome de função e uma lista de termos como argumentos.

O tipo `Clause` é usado para representar uma cláusula **Prolog** e pode ser uma cláusula simples ou uma cláusula com um corpo. O construtor `Simple` é usado para representar uma cláusula que consiste em apenas um termo, enquanto o construtor `:-` é usado para representar uma cláusula com um corpo, que consiste em uma cabeça (um termo) e o corpo da cláusula (uma lista de termos). O tipo **Prolog** definido neste projeto é simplesmente uma lista de cláusulas, que serão unificadas com a consulta realizada. Estas definições são exibidas na Figura 17.

```
unify :: Term -> Term -> Subst

substituteAll :: [(String, Term)] -> Term -> Term
```

Figura 18 – Cabeçalho das funções `unify` e `substituteAll`.

A substituição e a unificação são operações essenciais para o **Prolog**. Se `Just [(x1,t1), (x2,t2), ...]` é uma substituição, significa que a variável `x1` é substituída pelo termo `t1`, `x2` é substituído por `t2` e assim por diante. Quando possível, a unificação entre dois termos gera uma ou mais substituições a serem realizadas pela função `substituteAll`, que substitui uma variável por um termo ou um átomo, em um outro termo passado como argumento, como exibido na Figura 18.

Antes de invocar a função `interpret` e unificar a consulta com o programa, aplicamos a conversão alfa, que substitui as variáveis utilizadas na consulta por outra nova. Isso garante que não haverá conflitos com outras variáveis já existentes no programa. O tipo `Subst` representa uma substituição, que é uma associação entre variáveis e termos. Este, é uma lista de tuplas de pares `String` e `Term`, onde a `String`, que

representa uma variável é substituída pelo termo. Se `Nothing` for retornado, significa que não há nenhuma substituição possível.

A Função `interpret` exibida na Figura 19, retorna uma lista de tuplas `(String,Termo)` que simboliza uma substituição realizada, é responsável por percorrer o programa **Prolog** unificando as cláusulas que deram match com o termo. Caso a lista `query` possua um ou mais elementos, a unificação será requisitada através da função auxiliar `interpretClause`, para localizar e retornar uma ou mais substituições, que unifiquem o cabeçalho da cláusula correspondente com o termo consultado. Se essa substituição (este unificador) não existir, `Nothing` será retornado. A função `query` é uma sub-lista de todas as cláusulas no programa que podem potencialmente unificar com o termo fornecido, utilizando a função `match` para filtrar estas cláusulas potencialmente unificáveis (as que possuem pelo menos a cabeça da cláusula unificável com o termo consultado), caso não exista nenhuma, `False` será retornado.

Se ao menos uma substituição for retornada, `interpretBody` será chamado recursivamente no corpo da cláusula correspondente, com a substituição `subst` aplicado a cada termo do corpo. Se `interpretBody` retornar `Nothing`, a função `interpret` também retorna. Caso contrário, `subst'` é a substituição resultante da chamada recursiva para `interpretBody`, e `subst` e `subst'` são concatenados e retornados como a substituição final.

```
interpret :: Prolog -> Term -> Subst
interpret prog term = let substs = mapMaybe (interpretClause term) query
  in case substs of
    [] -> Nothing
    _ -> Just (concat substs)
interpretClause t c = case unify (headOf c) t of
  Nothing -> Nothing
  Just subst -> case interpretBody (map (substituteAll subst) (bodyOf c)) of
    Nothing -> interpret (tail prog) t
    Just subst' -> Just (subst ++ subst')
where
  interpretBody [] = Just []
  interpretBody (t:ts) = case interpret prog t of
    Nothing -> Nothing
    Just subst'' -> case interpretBody (map (substituteAll subst'') ts) of
      Nothing -> Nothing
      Just subst''' -> Just (subst'' ++ subst''')
```

Figura 19 – Função principal da DSL, implementa a resolução SLD do algoritmo do prolog.

Como consequência, a função `interpret` implementa o mecanismo de Backtracking por meio de seu uso de correspondência de padrões e recursão. Se uma consulta tiver várias cláusulas que correspondam ao cabeçalho da consulta realizada, apenas a primeira cláusula correspondente será selecionada para ser interpretada. No entanto, se essa cláusula resultar em uma falha (ou seja, retornar `Nothing`), a função retrocederá e tentará interpretar a próxima cláusula.

```

queryResult :: Prolog -> Term -> [(String, String)]
interpret   :: Prolog -> Term -> Subst

```

Figura 20 – Cabeçalho das funções `queryResult` e `interpret`.

A função `queryResult`, resulta em uma lista de tuplas de Strings, como exibido nas Figuras 21 e 22, sendo a primeira String uma variável, e a segunda o átomo unificado na variável, ou seja, a resposta para a consulta, como pode ser visto na Figura 20. Ela recebe a lista de substituições da função `interpret`. Como esta concatena as substituições de todas as variáveis utilizadas, incluindo as variáveis internas do programa, e as retorna para a função `queryResult`, filtra-se através da função `filterVars` apenas as variáveis de interesse para a resposta da consulta, ou seja, variáveis presentes no termo consultado, que são obtidas através da função `varsInTerm`. Também é na função `queryResult` que a conversão alfa é aplicada, através da função `rename`, e o resultado final é desencapsulado pela função `result`.

```

GHCi, version 9.0.1: https://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main
Ok, one module loaded.
ghci> queryResult myExample2 (Func "likes" [Atom "prolog", Var "Y"])
[("Y", "max")]
ghci>

```

Figura 21 – Resultado para o exemplo da Figura 16.

```

GHCi, version 9.0.1: https://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main
Ok, one module loaded.
ghci> queryResult myExample2 (Func "likes" [Atom "haskell", Var "Y"])
[("Y", "claire")]
ghci>

```

Figura 22 – Resultado para o mesmo exemplo da Figura 16, passando um argumento diferente.

Para evitar a necessidade de reescrever a consulta novamente no interpretador GHCi, podemos definir a consulta no programa como uma função principal ("main"). Isso permite que a consulta seja executada diretamente ao digitar `main` no interpretador, modificando as entradas no próprio programa. Além disso, também é possível criar uma função que testa várias consultas ao mesmo tempo. Uma abordagem útil quando você tem várias consultas para testar. No primeiro apêndice, você pode encontrar estas funcionalidades e outros exemplos de programas escritos na sintaxe da DSL desenvolvida. No segundo, você encontrará imagens da execução de consultas realizadas nos programas do primeiro apêndice e as respostas obtidas.

4 CONCLUSÃO

Neste trabalho foi desenvolvida uma DSL, disponível em https://github.com/ar1vit0r/Prolog_Library, que apresenta uma abordagem para a interpretação de programas escritos em **Prolog** dentro da linguagem **Haskell**. O desenvolvimento do interpretador permitiu explorar conceitos fundamentais da linguagem, tais como unificação, cláusulas e termos, e a forma como eles são manipulados, até um resolvente ser gerado.

Além disso, o uso da linguagem funcional **Haskell** permitiu expandir os conhecimentos sobre conceitos de programação funcional, como recursividade e a correspondência de padrões. A Recursividade é uma técnica comum em programação, porém melhor aproveitada em **Haskell**, para as mais variadas tarefas, mas principalmente para percorrer estruturas de dados, enquanto a correspondência de padrões (do inglês, Pattern Matching) é uma forma poderosa de desconstruir as estruturas de dados em suas partes constituintes e executar diferentes ações com base em suas estruturas.

Estas, são algumas das vantagens da programação funcional na construção de interpretadores de linguagens de programação. O desenvolvimento desta DSL também salientou a importância de definir tipos de dados claros e funções auxiliares, pois apesar da programação funcional nos resultar em um código enxuto e curto, apresenta uma alta complexidade, por causa, por exemplo, dos inúmeros loops de recursão ou da forma como as funções são valores de primeira categoria, também pode confundir um pouco a primeira vista.

Esta implementação usa listas como estrutura de dados para representar substituições, por que o **Haskell** facilita muito sua manipulação através de suas inúmeras bibliotecas, porém, elas podem ser ineficientes para consultas grandes ou programas com muitas variáveis. Um método mais eficiente seria usar HashMap's ou outra estrutura de dados que forneça pesquisas e atualizações mais rápidas.

Como dificuldades encontradas ao longo do desenvolvimento, posso citar a dificuldade em encontrar referências sobre o assunto, principalmente, fora da notação matemática amplamente utilizada na maioria das bibliografias encontradas.

É importante destacar que o interpretador implementado neste projeto possui funcionalidades limitadas e não é capaz de lidar com todos os recursos oferecidos pelo **Prolog**. Recursos avançados, como regras recursivas, predicados integrados e estruturas de controle mais avançadas não são suportados pelo interpretador. Note também que as cláusulas no programa, nem o termo na consulta, são validados, ou seja, se alguma cláusula do programa tiver sintaxe incorreta, um erro de execução será produzido.

Logo, este projeto pode ser visto como uma introdução a implementação do **Prolog** e à programação funcional em **Haskell**, e pode servir como base para futuros trabalhos de implementação de interpretadores mais complexos. Outras melhorias no interpretador podem incluir a implementação de recursos mais avançados, como o mecanismo de corte, negação como falha e estruturas de controle como loops if-then-else e while.

REFERÊNCIAS

- ANTOY, S. Evaluation strategies for functional logic programming. **Journal of Symbolic Computation**, [S.l.], v.40, n.1, p.875–903, July 2005.
- ANTOY, S.; HANUS, M. Functional logic programming. **Commun. ACM**, [S.l.], v.53, n.4, p.74–85, 2010.
- DINUCCI, A. et al. Introdução à lógica proposicional estoica. **Prometheus-Journal of Philosophy**, [S.l.], 2016.
- FERNÁNDEZ, M. **Programming Languages and Operational Semantics**. [S.l.]: College Publications, 2004. I–VI, 1–142p. (Texts in computing, v.1).
- GINSBERG, M. L. Dynamic backtracking. **Journal of artificial intelligence research**, [S.l.], v.1, p.25–46, 1993.
- HANUS, M.; KUCHEN, H.; MORENO-NAVARRO, J. J. Curry: A truly functional logic language. In: ILPS, 1995. **Proceedings...** [S.l.: s.n.], 1995. v.95, n.5, p.95–107.
- MARTELLI, A.; MONTANARI, U. An efficient unification algorithm. **ACM Transactions on Programming Languages and Systems (TOPLAS)**, [S.l.], v.4, n.2, p.258–282, 1982.
- MONARD, M. C.; BARANAUSKAS, J. A. Indução de regras e árvores de decisão. **Sistemas Inteligentes-Fundamentos e Aplicações**, [S.l.], v.1, p.115–139, 2003.
- ROUSSEL, P. Prolog: Manuel de Reference et d’Utilisation Groupe d’Intelligence Artificielle. **UER de Luminy, Universit6 d’Aix-Marseille II**, [S.l.], 1975.
- SNYDER, W. Unification theory. **Handbook of automated reasoning**, [S.l.], v.1, p.447–533, 2001.
- THOMPSON, S. **The Craft of Functional Programming**. [S.l.]: Addison-Wesley, 1996.
- WIELEMAKER, J.; SCHRIJVERS, T.; TRISKA, M.; LAGER, T. SWI-Prolog. **Theory and Practice of Logic Programming**, [S.l.], 2012.

Apêndices

APÊNDICE A – Exemplos

```

1
2 -- main to execute all the examples
3 main :: IO ()
4 main = do
5     example
6     example1
7     example2
8     example3
9     example4
10    example5
11    example6
12    example7
13
14 example :: IO ()
15 example = print (queryResult myExample (Func "mae" [Var "Y", Atom "ari_vitor"]))
16
17 example1 :: IO ()
18 example1 = print (queryResult myExample1 (Func "consulta" [Var "Q"]))
19
20 example2 :: IO ()
21 example2 = print (queryResult myExample2 (Func "parent" [Atom "arne", Var "X"]))
22
23 example3 :: IO ()
24 example3 = print (queryResult myExample3 (Func "likes" [Var "Y", Atom "prolog"]))
25
26 example4 :: IO ()
27 example4 = print (queryResult myExample4 (Func "professor" [Var "Y", Atom "charlie"]))
28
29 example5 :: IO ()
30 example5 = print (queryResult myExample5 (Func "gives" [Var "ALGUEM1", Atom "chocolate",
31 Var "ALGUEM2"]))
32
33 example6 :: IO ()
34 example6 = print (queryResult myExample6 (Func "woman" [Var "X"]))

```

```

35
36 example7 :: IO ()
37 example7 = print (queryResult myExample7 (Func "king" [Atom "menelaus", Var "X", Var "Y"]))
38
39 -----
40 --Exemplo de programa Prolog-----
41 -----
42 myExample :: Prolog
43 myExample = [
44     Simple (Func "progenitor" [Atom "joao", Atom "ari" ]),
45     Simple (Func "progenitor" [Atom "vitoria", Atom "ari" ]),
46     Simple (Func "progenitor" [Atom "paulina", Atom "janeti" ]),
47     Simple (Func "progenitor" [Atom "olicio", Atom "janeti" ]),
48     Simple (Func "progenitor" [Atom "janeti", Atom "ari_vitor" ]),
49     Simple (Func "progenitor" [Atom "ari", Atom "ari_vitor" ]),
50     Simple (Func "progenitor" [Atom "ari", Atom "ariel" ]),
51     Simple (Func "progenitor" [Atom "janeti", Atom "ariel" ]),
52     Simple (Func "sexo" [Atom "paulina", Atom "feminino"]),
53     Simple (Func "sexo" [Atom "vitoria", Atom "feminino"]),
54     Simple (Func "sexo" [Atom "janeti", Atom "feminino"]),
55     Simple (Func "sexo" [Atom "ari", Atom "masculino"]),
56     Simple (Func "sexo" [Atom "joao", Atom "masculino"]),
57     Simple (Func "sexo" [Atom "olicio", Atom "masculino"]),
58     Simple (Func "sexo" [Atom "ari_vitor", Atom "masculino"]),
59     Simple (Func "sexo" [Atom "ariel", Atom "masculino"]),
60     Func "mae" [Var "X", Var "Y"] :- [Func "progenitor" [Var "X", Var "Y"],
61     Func "sexo" [Var "X", Atom "feminino"]],
62     Func "pai" [Var "X", Var "Y"] :- [Func "progenitor" [Var "X", Var "Y"],
63     Func "sexo" [Var "X", Atom "masculino"]]]
64
65 myExample1 :: Prolog
66 myExample1 = [
67     Simple (Func "irmao" [Atom "andre", Atom "joao"]),
68     Func "consulta" [Var "X"] :- [Func "irmao" [Atom "andre", Var "X" ]]]
69
70 --Modern Compiler Design example page 613
71 myExample2 :: Prolog
72 myExample2 = [
73     Simple (Func "parent" [Atom "arne", Atom "james"]),
74     Simple (Func "parent" [Atom "arne", Atom "sachiko"]),
75     Simple (Func "parent" [Atom "koos", Atom "rivka"]),

```

```

76     Simple (Func "parent" [Atom "sachiko", Atom "rivka"]),
77     Simple (Func "parent" [Atom "truitje ", Atom "koos"])]
78
79     --Maribel example page 171
80 myExample3 :: Prolog
81 myExample3 = [
82     Simple (Func "based" [Atom "prolog", Atom "logic"]),
83     Simple (Func "based" [Atom "haskell", Atom "maths"]),
84     Simple (Func "likes" [Atom "max", Atom "logic"]),
85     Simple (Func "likes" [Atom "claire", Atom "maths"]),
86     Func "likes" [Var "X", Var "P"] :- [Func "based" [Var "P", Var "Y"],
87     Func "likes" [Var "X", Var "Y" ]]]
88
89 myExample4 :: Prolog
90 myExample4 = [
91     Simple (Func "studies" [Atom "charlie", Atom "csc135"]),
92     Simple (Func "studies" [Atom "olivia", Atom "csc135"]),
93     Simple (Func "studies" [Atom "jack", Atom "csc131"]),
94     Simple (Func "studies" [Atom "arthur", Atom "csc131"]),
95     Simple (Func "teaches" [Atom "kirke", Atom "csc135"]),
96     Simple (Func "teaches" [Atom "collins", Atom "csc131"]),
97     Simple (Func "teaches" [Atom "collins", Atom "csc171"]),
98     Simple (Func "teaches" [Atom "juniper", Atom "csc134"]),
99     Func "professor" [Var "X", Var "Y"] :- [Func "teaches" [Var "X", Var "Z"],
100     Func "studies" [Var "Y", Var "Z" ]]]
101
102 myExample5 :: Prolog
103 myExample5 = [
104     Simple (Func "gives" [Atom "john", Atom "chocolate", Atom "jane"])]
105
106 myExample6 :: Prolog
107 myExample6 = [
108     Simple (Func "woman" [Atom "mia"]),
109     Simple (Func "woman" [Atom "jody"]),
110     Simple (Func "woman" [Atom "yolanda"]),
111     Simple (Func "loves" [Atom "vincent", Atom "mia"]),
112     Simple (Func "loves" [Atom "marcellus", Atom "mia"]),
113     Simple (Func "loves" [Atom "pumpkin", Atom "honey_bunny"]),
114     Simple (Func "loves" [Atom "honey_bunny", Atom "pumpkin"]),
115     Func "loves" [Var "X", Var "Y"] :- [Func "loves" [Var "X", Var "Y"],
116     Func "woman" [Var "Y"]],

```

```

117     Func "jealous" [Var "X", Var "Y"] :- [Func "loves" [Var "X", Var "Z"],
118     Func "loves" [Var "Y", Var "Z" ]]]
119
120 myExample7 :: Prolog
121 myExample7 = [
122     Simple (Func "character" [Atom "priam", Atom "iliad" ]),
123     Simple (Func "character" [Atom "hecuba", Atom "iliad" ]),
124     Simple (Func "character" [Atom "achilles", Atom "iliad" ]),
125     Simple (Func "character" [Atom "agamemnon", Atom "iliad"]),
126     Simple (Func "character" [Atom "patroclus", Atom "iliad" ]),
127     Simple (Func "character" [Atom "hector", Atom "iliad" ]),
128     Simple (Func "character" [Atom "andromache", Atom "iliad"]),
129     Simple (Func "character" [Atom "rhesus", Atom "iliad" ]),
130     Simple (Func "character" [Atom "ulysses", Atom "iliad" ]),
131     Simple (Func "character" [Atom "menelaus", Atom "iliad"]),
132     Simple (Func "character" [Atom "helen", Atom "iliad" ]),
133     Simple (Func "character" [Atom "ulysses", Atom "odyssey"]),
134     Simple (Func "character" [Atom "penelope", Atom "odyssey"]),
135     Simple (Func "character" [Atom "telemachus", Atom "odyssey"]),
136     Simple (Func "character" [Atom "laertes", Atom "odyssey"]),
137     Simple (Func "character" [Atom "nestor", Atom "odyssey"]),
138     Simple (Func "character" [Atom "menelaus", Atom "odyssey"]),
139     Simple (Func "character" [Atom "helen", Atom "odyssey"]),
140     Simple (Func "character" [Atom "hermione", Atom "odyssey"]),
141     Simple (Func "male" [Atom "priam"]),
142     Simple (Func "male" [Atom "achilles" ]),
143     Simple (Func "male" [Atom "agamemnon"]),
144     Simple (Func "male" [Atom "patroclus"]),
145     Simple (Func "male" [Atom "hector"]),
146     Simple (Func "male" [Atom "rhesus"]),
147     Simple (Func "male" [Atom "ulysses"]),
148     Simple (Func "male" [Atom "menelaus"]),
149     Simple (Func "male" [Atom "telemachus"]),
150     Simple (Func "male" [Atom "laertes"]),
151     Simple (Func "male" [Atom "nestor"]),
152     Simple (Func "female" [Atom "hecuba"]),
153     Simple (Func "female" [Atom "andromache"]),
154     Simple (Func "female" [Atom "helen"]),
155     Simple (Func "female" [Atom "penelope"]),
156     Simple (Func "father" [Atom "priam", Atom "hector"]),
157     Simple (Func "father" [Atom "laertes", Atom "ulysses"]),

```

```

158 Simple (Func "father" [Atom "atreus", Atom "menelaus"]),
159 Simple (Func "father" [Atom "menelaus", Atom "hermione"]),
160 Simple (Func "father" [Atom "ulysses", Atom "telemachus"]),
161 Simple (Func "mother" [Atom "hecuba", Atom "hector"]),
162 Simple (Func "mother" [Atom "penelope", Atom "telemachus"]),
163 Simple (Func "mother" [Atom "helen", Atom "hermione"]),
164 Simple (Func "king" [Atom "ulysses", Atom "ithaca", Atom "achaeon"]),
165 Simple (Func "king" [Atom "menelaus", Atom "sparta", Atom "achaeon"]),
166 Simple (Func "king" [Atom "nestor", Atom "pylos", Atom "achaeon"]),
167 Simple (Func "king" [Atom "agamemnon", Atom "argos", Atom "achaeon"]),
168 Simple (Func "king" [Atom "priam", Atom "troy", Atom "trojan" ]),
169 Simple (Func "king" [Atom "rhesus", Atom "thrace", Atom "trojan" ])]

```

APÊNDICE B – Consultas

```
GHCI, version 9.0.1: https://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main
Ok, one module loaded.
ghci> queryResult myExample1 (Func "consulta" [Var "Q"])
[("Q","joao")]
ghci> █
```

Figura 23 – Consulta realizada no exemplo 1.

```
GHCI, version 9.0.1: https://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main
Ok, one module loaded.
ghci> queryResult myExample2 (Func "parent" [Atom "arne", Var "X"])
[("X","james"),("X","sachiko")]
ghci> █
```

Figura 24 – Consulta realizada no exemplo 2.

```
GHCI, version 9.0.1: https://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main
Ok, one module loaded.
ghci> queryResult myExample3 (Func "likes" [Var "Y", Atom "prolog"])
[("Y","max")]
ghci> █
```

Figura 25 – Consulta realizada no exemplo 3.

```

GHCi, version 9.0.1: https://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main
Ok, one module loaded.
ghci> queryResult myExample4 (Func "professor" [Var "Y", Atom "charlie"])
[("Y","kirke")]
ghci>

```

Figura 26 – Consulta realizada no exemplo 4.

```

GHCi, version 9.0.1: https://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main
Ok, one module loaded.
ghci> queryResult myExample5 (Func "gives" [Var "ALGUEM1", Atom "chocolate", Var "ALGUEM2"])
[("ALGUEM1","john"),("ALGUEM2","jane")]
ghci>

```

Figura 27 – Consulta realizada no exemplo 5.

```

GHCi, version 9.0.1: https://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main
Ok, one module loaded.
ghci> queryResult myExample6 (Func "woman" [Var "X"])
[("X","mia"),("X","jody"),("X","yolanda")]
ghci>

```

Figura 28 – Consulta realizada no exemplo 6.

```

GHCi, version 9.0.1: https://www.haskell.org/ghc/  :? for help
[1 of 1] Compiling Main
Ok, one module loaded.
ghci> queryResult myExample7 (Func "king" [Atom "menelaus", Var "X", Var "Y"])
[("X","sparta"),("Y","achaeon")]
ghci>

```

Figura 29 – Consulta realizada no exemplo 7.